Solver 서비스 아키텍처 분석

개요

Solver 서비스는 Omni 네트워크의 SolverNet 참조 구현으로, 크로스체인 거래 최적화와 유동성 관리를 담당하는 핵심 컴포넌트입니다. 이 서비스는 사용자의 크로스체인 주문을 효율적으로 처리하고, 다양한 체인 간의 자산 이동을 최적화합니다.

핵심 아키텍처

1. 전체 구조

```
┌─────────────────────────────────────────────────────────────┐
│ Solver Service │
├──────────────┬──────────────┬──────────────┬──────────┤
│ Event Stream │ Order Proc │ Rebalancing │ API │
│ (감지/수집) │ (주문처리) │ (유동성관리) │ (외부IF) │
├──────────────┼──────────────┼──────────────┼──────────┤
│ Job Queue │ Price Oracle │ Fund Thresh │ Monitor │
│ (작업관리) │ (가격정보) │ (임계값관리) │ (메트릭) │
└──────────────┴──────────────┴──────────────┴──────────┘
```

2. 주요 컴포넌트

App 구조체 (`solver/app/app.go`)

Solver 서비스의 메인 진입점으로, 다음과 같은 핵심 기능을 초기화합니다:

```go
func Run(ctx context.Context, cfg Config) error {
// 1. 모니터링 서버 시작
monitorChan := serveMonitoring(cfg.MonitoringAddr)
// 2. Portal Registry 생성
portalReg, err := makePortalRegistry(ctx, cfg.Network, cfg.RPCEndpoints)
// 3. 네트워크 설정 및 백엔드 초기화
backends, err := ethbackend.BackendsFromNetwork(ctx, network, cfg.RPCEndpoints,
// 4. 데이터베이스 및 작업 큐 초기화
jobDB, err := job.New(db)
// 5. 이벤트 처리 시작
err = startProcessingEvents(ctx, network, xprov, jobDB, ...)
// 6. 리밸런싱 시작
if err := rebalance.Start(ctx, network, ...); err != nil {
log.Warn(ctx, "Failed to start rebalancing [BUG]", err)
}
// 7. API 서버 시작
apiChan, apiCancel := serveAPI(cfg.APIAddr, handlers...)
}
```

설정 구조체 (`solver/app/config.go`)

```go
type Config struct {
RPCEndpoints xchain.RPCEndpoints // 체인별 RPC 엔드포인트
Network netconf.ID // 네트워크 식별자
MonitoringAddr string // 모니터링 서버 주소
APIAddr string // API 서버 주소
SolverPrivKey string // Solver 개인키 파일 경로
DBDir string // 데이터베이스 디렉토리
Tracer tracer.Config // 트레이싱 설정
CoinGeckoAPIKey string // 가격 정보 API 키
}
```

주문 처리 시스템

1. 주문 타입 정의 (`solver/app/types.go`)

```go
type Order struct {
ID OrderID // 주문 고유 식별자
Offset uint64 // 블록체인 오프셋
SourceChainID uint64 // 소스 체인 ID
Status solvernet.OrderStatus // 주문 상태
UpdatedBy common.Address // 업데이트한 주소
pendingData PendingData // 대기 중 주문 데이터
filledData FilledData // 완료된 주문 데이터
```

```
}

type PendingData struct {
MinReceived []bindings.IERC7683Output // 최소 수령 금액
DestinationSettler common.Address // 목적지 정산자
DestinationChainID  uint64  // 목적지 체인 ID
FillOriginData []byte  // 채우기 원본 데이터
MaxSpent []bindings.IERC7683Output // 최대 지출 금액
}
```

2. 주문 상태 전환

```
Pending → Filled → Claimed
↓
Rejected
```

각 상태별 처리 로직:

- **Pending**: 주문 검증 및 채우기 실행
- **Filled**: 주문 완료 후 클레임 처리
- **Rejected**: 거부된 주문 처리
- **Claimed**: 최종 정산 완료

3. 이벤트 처리기 (`solver/app/processor.go`)

```go
func newEventProcFunc(deps procDeps, chainID uint64) eventProcFunc {
return func(ctx context.Context, e Event) error {
// 1. 주문 조회
order, found, err := deps.GetOrder(ctx, chainID, e.OrderID)
// 2. 상태 전환 검증
if !e.Status.ValidTarget(order.Status) {
return errors.New("invalid order transition")
}
// 3. 상태별 처리
switch e.Status {
case solvernet.StatusPending:
// 거부 여부 확인
if didReject, err := maybeReject(); err != nil {
return err
} else if didReject {
break
}
// 주문 채우기
if err := deps.Fill(ctx, order); err != nil {
return errors.Wrap(err, "fill order")
}
case solvernet.StatusFilled:
```

```go
// 주문 클레임
if err := deps.Claim(ctx, order); err != nil {
return errors.Wrap(err, "claim order")
}
}
return nil
}
}
```

유동성 관리 시스템

1. 리밸런싱 구조 (`solver/rebalance/rebalance.go`)

```go
func rebalanceCCTPOnce(
ctx context.Context,
db *cctpdb.DB,
network netconf.Network,
pricer tokenpricer.Pricer,
backends ethbackend.Backends,
solver common.Address,
) {
for _, chain := range network.EVMChains() {
// 1. 잉여 토큰을 USDC로 스왑
```

```go
if err := swapSurplusOnce(ctx, backends, chain.ID, solver); err != nil {
log.Warn(ctx, "Failed to swap surplus", err)
}
// 2. 부족분을 잉여 USDC로 채우기
if err := fillDeficitOnce(ctx, pricer, backends, chain.ID, solver); err != nil
log.Warn(ctx, "Failed to fill deficit", err)
}
// 3. 남은 잉여 USDC를 다른 체인으로 전송
if err := sendSurplusOnce(ctx, db, network, pricer, backends, chain.ID, solver)
log.Warn(ctx, "Failed to send surplus", err)
}
}
}
```

2. 유동성 임계값 관리

```go
var (
minSend = bi.Dec6(1000) // 최소 전송량: 1,000 USDC
maxSend = bi.Dec6(10000) // 최대 전송량: 10,000 USDC
)
```

3. 스왑 로직

- **잉여 토큰 → USDC**: Uniswap V3를 통한 자동 스왑

- **USDC → 부족 토큰**: 필요시 USDC를 다른 토큰으로 스왑
- **체인 간 USDC 전송**: CCTP(Circle's Cross-Chain Transfer Protocol) 사용

API 시스템

1. 핵심 엔드포인트 (`solver/app/handler.go`)

```
// 주문 검증 엔드포인트
func newCheckHandler(checkFunc checkFunc, traceFunc traceFunc) Handler {
return Handler{
Endpoint: "/api/v1/check",
HandleFunc: func(ctx context.Context, request any) (any, error) {
req := request.(*types.CheckRequest)
err := checkFunc(ctx, *req)
if r := new(RejectionError); errors.As(err, &r) {
return types.CheckResponse{
Rejected: true,
RejectCode: r.Reason,
RejectReason: r.Reason.String(),
RejectDescription: errors.Format(r.Err),
}, nil
}
return types.CheckResponse{Accepted: true}, nil
},
```

```
}

}
```

2. API 타입 정의 (`solver/types/api.go`)

```
type CheckRequest struct {

SourceChainID  uint64  `json:"sourceChainId"`  // 소스 체인 ID

DestinationChainID  uint64  `json:"destChainId"`  // 목적지 체인 ID

FillDeadline  uint32  `json:"fillDeadline"`  // 채우기 마감시간

Calls []Call `json:"calls"`  // 실행할 호출들

Expenses []Expense `json:"expenses"`  // 비용 정보

Deposit AddrAmt `json:"deposit"`  // 예치 정보

Debug  bool  `json:"debug"`  // 디버그 모드

}


type CheckResponse struct {

Accepted  bool  `json:"accepted"`  // 승인 여부

Rejected  bool  `json:"rejected"`  // 거부 여부

RejectCode RejectReason `json:"rejectCode"`  // 거부 코드

RejectReason  string  `json:"rejectReason"`  // 거부 사유

RejectDescription  string  `json:"rejectDescription"`  // 거부 설명

Trace  map[string]any `json:"trace"`  // 트레이스 정보
```

```
}
```

가격 시스템

1. 가격 구조체 (solver/types/price.go)

```
type Price struct {
Price *big.Rat // 가격 비율 (expense/deposit)
Deposit tokens.Asset // 예치 토큰
Expense tokens.Asset // 비용 토큰
}

// 수수료 적용
func (p Price) WithFeeBips(fee  int64) Price {
clone := p
if  fee > 0 {
feeRat := big.NewRat(10_000, 10_000+fee)
clone.Price = new(big.Rat).Mul(clone.Price, feeRat)
}
return  clone
}

// 예치 금액을 비용 금액으로 변환
```

```go
func (p Price) ToExpense(depositAmount *big.Int) *big.Int {
rebasedDeposit := bi.Rebase(depositAmount, p.Deposit.Decimals, p.Expense.Decimal
return  bi.MulRatFloor(rebasedDeposit, p.Price)
}
```

2. 가격 오라클 통합

- **CoinGecko API**: 실시간 토큰 가격 정보
- **Uniswap V3**: 온체인 가격 정보
- **가격 캐싱**: 성능 최적화를 위한 가격 정보 캐싱

거부 시스템

1. 거부 사유 (`solver/types/reject.go`)

```go
type RejectReason uint8

const (
RejectNone RejectReason = 0  // 거부 없음
RejectDestCallReverts RejectReason = 1  // 목적지 호출 실패
RejectInvalidDeposit RejectReason = 2  // 잘못된 예치
RejectInvalidExpense RejectReason = 3  // 잘못된 비용
RejectInsufficientDeposit RejectReason = 4  // 예치 부족
RejectInsufficientInventory RejectReason = 5  // 재고 부족
RejectUnsupportedDeposit RejectReason = 6  // 지원하지 않는 예치 토큰
```

```
RejectUnsupportedExpense RejectReason = 7  // 지원하지 않는 비용 토큰

RejectUnsupportedDestChain RejectReason = 8  // 지원하지 않는 목적지 체인

RejectUnsupportedSrcChain RejectReason = 9  // 지원하지 않는 소스 체인

RejectExpenseOverMax RejectReason = 11  // 비용 한도 초과

RejectExpenseUnderMin RejectReason = 12  // 비용 최소값 미달

RejectCallNotAllowed RejectReason = 13  // 허용되지 않는 호출

)
```

모니터링 및 메트릭

1. 프로메테우스 메트릭

- **processedEvents**: 처리된 이벤트 수
- **rejectedOrders**: 거부된 주문 수
- **statusOffset**: 상태별 오프셋 추적

2. 로깅 시스템

```
log.Debug(ctx, "Processing order event",

"order_id", order.ID.String(),

"offset", order.Offset,

"status", order.Status,

)
```

데이터베이스 시스템

1. Job 데이터베이스 (`solver/job/types.go`)

- **작업 큐**: 비동기 주문 처리를 위한 작업 관리
- **상태 추적**: 주문 상태 변경 이력 관리
- **재시작 지원**: 서비스 재시작 시 미완료 작업 복구

2. 커서 관리

- **블록 높이 추적**: 각 체인별 처리된 마지막 블록 높이
- **중복 처리 방지**: 동일한 이벤트의 중복 처리 방지
- **재동기화**: 네트워크 문제 시 자동 재동기화

보안 및 안정성

1. 개인키 관리

```
if  cfg.SolverPrivKey == "" {
return  errors.New("private key not set")
}
privKey, err := ethcrypto.LoadECDSA(cfg.SolverPrivKey)
```

2. 승인 관리

- **토큰 승인**: 각 체인의 Outbox 컨트랙트에 대한 토큰 승인
- **승인 확인**: 거래 실행 전 충분한 승인 여부 확인

3. 에러 처리

- **재시도 로직**: 일시적 네트워크 오류에 대한 자동 재시도
- **회로 차단기**: 연속적인 실패 시 서비스 보호
- **그레이스풀 셧다운**: 안전한 서비스 종료

확장성 고려사항

1. 멀티체인 지원

- **EVM 체인**: Ethereum, Polygon, Arbitrum 등
- **SVM 체인**: Solana 지원 (`solver/app/svmproc.go`)
- **새로운 체인**: 플러그인 방식으로 새로운 체인 추가 가능

2. 성능 최적화

- **병렬 처리**: 체인별 독립적인 이벤트 처리
- **배치 처리**: 여러 주문의 일괄 처리
- **캐싱**: 가격 정보 및 메타데이터 캐싱

이 아키텍처는 Omni 네트워크의 크로스체인 거래를 효율적으로 처리하면서도 높은 안정성과 확장성을 제공합니다.